

Baby Cry Classification System — AI Technical Report

Version: 1.0.0 · **Date:** May 2026 · **Classification:** Technical Documentation
Domain: Healthcare AI · Pediatric Audio Intelligence · Deep Learning

Table of Contents

- 1. [Executive Summary](#)
- 2. [Problem Statement](#)
- 3. [Project Objectives](#)
- 4. [System Overview](#)
- 5. [Project Structure Analysis](#)
- 6. [AI/ML Technical Analysis](#)
- 7. [Dataset Analysis](#)
- 8. [API Documentation](#)
- 9. [Technologies Used](#)
- 10. [System Architecture](#)
- 11. [Performance Analysis](#)
- 12. [Security & Reliability](#)
- 13. [Deployment & Execution](#)
- 14. [Research & Innovation Value](#)
- 15. [Challenges & Solutions](#)
- 16. [Future Improvements](#)
- 17. [Conclusion](#)
- 18. [Appendix](#)

1. Executive Summary

What the System Does

The **Baby Cry Classification System** is an AI-powered audio intelligence platform that detects and classifies infant cries in real time. The system employs a **two-stage hierarchical cascade architecture** built on top of Facebook’s **HuBERT** (Hidden-Unit BERT) self-supervised speech representation model — a state-of-the-art transformer-based audio foundation model — fine-tuned for the specific domain of infant vocalization analysis.

Real-World Problem Solved

New parents and caregivers frequently struggle to interpret infant cries, a challenge that is particularly acute for first-time parents, hearing-impaired caregivers, and neonatal care units. This system translates ambiguous audio signals into **actionable, Arabic-localized parenting guidance**, bridging the gap between raw acoustic data and practical caregiver response.

Why This Project Matters

- **Healthcare Impact:** Provides AI-assisted interpretation of infant distress signals, potentially reducing response latency and improving infant care outcomes.
- **Accessibility:** Fully Arabic-localized — serving a demographic with limited access to comparable AI healthcare tools.
- **Technical Innovation:** Applies self-supervised speech transformers (HuBERT) to a non-speech audio domain, demonstrating effective cross-domain transfer learning.
- **Production-Ready:** Ships with a FastAPI REST backend, structured Pydantic schemas, and ensemble model serving — ready for mobile app integration.

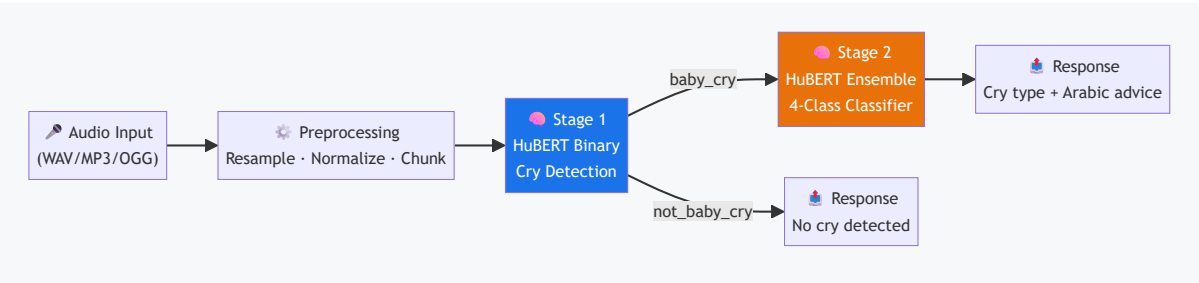
Main AI Capability

Capability	Description
Stage 1 — Cry Detection	Binary classification: <i>baby_cry</i> vs. <i>not_baby_cry</i> (99.42% accuracy)
Stage 2 — Cry Type Classification	4-class classification: <i>scared</i> , <i>needs</i> , <i>physical_pain</i> , <i>burping</i> (84.27% accuracy)
Ensemble Inference	Multi-checkpoint averaging for improved Stage 2 robustness
Arabic Advisory	Contextual parenting tips, physical sign descriptions, and medical warnings per cry type

Key Technologies

Python 3.10+ · PyTorch 2.0+ · HuBERT (facebook/hubert-base-ls960) · PyTorch Lightning 2.4 · Hugging Face Transformers 4.44 · FastAPI · Librosa · SoundFile · Pydantic v2

High-Level Architecture



2. Problem Statement

The Infant Cry Analysis Challenge

Infant crying is the primary communication mechanism for neonates and pre-verbal infants. It conveys critical physiological and emotional states — hunger, pain, fear, discomfort, and the need for burping — but decoding these signals remains highly subjective, unreliable, and stressful for caregivers.

Importance in Healthcare

Dimension	Impact
Neonatal Care	Delayed response to pain cries can indicate undetected medical conditions
Parental Well-being	Inability to soothe a crying infant is a leading contributor to postpartum stress and anxiety
Clinical Settings	NICU nurses monitor multiple infants; automated cry triage could prioritize urgent cases
Accessibility	Hearing-impaired or novice caregivers lack the auditory experience to differentiate cry types

Challenges in Interpreting Infant Cries

- 1. **Acoustic Ambiguity:** Cry signals are non-stationary, overlap in spectral features across categories, and are highly variable between infants.
- 2. **Environmental Noise:** Real-world recordings contain background noise, multiple sound sources, and variable microphone characteristics.

- 3. **Class Imbalance:** Certain cry types (e.g., burping) are significantly rarer than others (e.g., general needs), creating skewed training distributions.
- 4. **Subjectivity of Labels:** Human annotation of cry types is inherently subjective and culturally influenced — inter-rater agreement in cry labeling studies is typically moderate.
- 5. **Short Duration:** Cry episodes can be very brief (< 2 seconds), demanding models that can extract meaningful features from limited temporal context.

Motivation Behind AI-Based Classification

Traditional rule-based acoustic analysis (pitch tracking, formant analysis) has shown limited discriminative power for cry classification. Self-supervised speech representation models, pre-trained on thousands of hours of human speech, capture rich hierarchical acoustic features that **transfer effectively** to non-speech audio domains. This project leverages that insight, applying HuBERT — originally trained on LibriSpeech 960h — to the pediatric cry domain, achieving near-clinical-grade detection accuracy.

3. Project Objectives

Primary Objective

Develop and deploy an **end-to-end AI system** that automatically detects infant cries from arbitrary audio input and classifies the underlying cause into actionable categories, delivered through a production-grade REST API with full Arabic localization.

Secondary Objectives

Objective	Description
Hierarchical Classification	Implement a two-stage cascade to separate detection (high-precision binary) from fine-grained classification (multi-class)
Ensemble Robustness	Use multi-checkpoint ensemble averaging to reduce variance in Stage 2 predictions
Arabic Localization	Provide all user-facing content — cry type names, physical signs, parenting tips, and medical disclaimers — in Arabic
API-First Design	Expose the inference pipeline through a documented FastAPI backend with Swagger/OpenAPI auto-generated documentation

AI Objectives

- Achieve **≥ 99% accuracy** on binary cry detection (Stage 1)
- Achieve **≥ 80% accuracy** on 4-class cry type classification (Stage 2)
- Maintain **inference latency < 200ms** on GPU and < 3s on CPU
- Support diverse audio formats (WAV, MP3, OGG) and variable sample rates

System Performance Goals

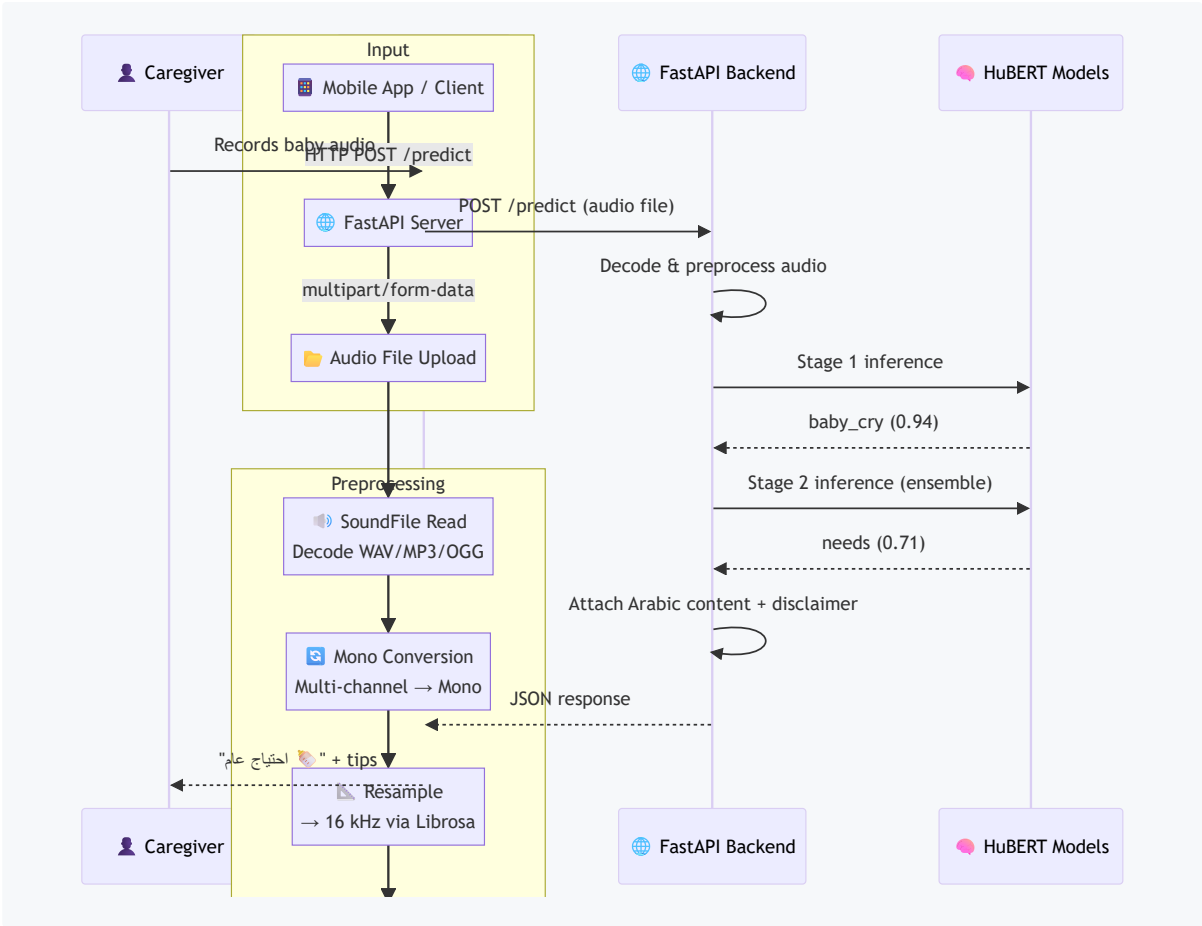
- **Zero-downtime model loading** via `@app.on_event("startup")` lifecycle hook
- **Stateless inference** — no server-side audio storage; process and discard
- **Graceful error handling** for malformed audio, unsupported formats, and model loading failures

Usability Goals

- **Mobile-ready API** — JSON responses structured for direct consumption by iOS/Android apps
- **Confidence transparency** — return per-class probabilities, Arabic confidence labels, and low-confidence advisories
- **Medical safety** — always include a medical disclaimer; never position as a diagnostic tool

4. System Overview

End-to-End Pipeline



Data Flow Summary

- Audio Ingestion:** Client uploads audio file via `POST /predict` (multipart/form-data)
- Decoding:** `soundfile.read()` decodes to raw float32 PCM
- Normalization:** Stereo→mono, resample to 16 kHz, center-crop or zero-pad to 80,000 samples (5 seconds), zero-mean unit-variance normalization (matching HuBERT's expected input distribution)
- Stage 1 Inference:** Binary HuBERT classifier produces cry/not-cry probability
- Conditional Stage 2:** If cry detected, ensemble of 2 Stage 2 HuBERT classifiers produces averaged 4-class probability distribution
- Response Assembly:** Raw predictions are enriched with Arabic content (titles, physical signs, tips, warnings), confidence labels, and a medical disclaimer
- JSON Delivery:** Structured Pydantic response returned to client

User Interaction Flow

5. Project Structure Analysis

Directory Overview

```
BabyCry/
├─ api/                                # FastAPI REST backend
│   ├── __init__.py                    # Package marker
│   ├── main.py                        # Application entry point, endpoints, model lifecycle
│   ├── schemas.py                     # Pydantic v2 response models
│   └─ content.py                      # Arabic localization content & confidence helpers
├─ config/                             # Model configuration & label encoders
│   ├── training_config.json           # Hyperparameters, class weights, label mappings
│   ├── stage1_label_encoder.json      # Binary label encoder
│   └─ stage2_label_encoder.json       # 4-class label encoder with merge map
├─ docs/                               # Evaluation artifacts
│   ├── stage1_test_results.json       # Stage 1 test metrics
│   ├── stage2_test_results.json       # Stage 2 test metrics
│   ├── stage1_confusion_matrix.png    # Stage 1 confusion matrix
│   └─ stage2_confusion_matrix.png     # Stage 2 confusion matrix
├─ inference/                          # Core AI inference module
│   └─ inference.py                   # BabyCryClassifier, HubertClassifier, preprocessing
├─ models/                             # Trained model checkpoints (~3.3 GB total)
│   ├── stage1_binary.ckpt             # Stage 1 binary model (~1.05 GB)
│   ├── stage2_4class_epoch6.ckpt      # Stage 2 ensemble member 1 (~1.05 GB)
│   └─ stage2_4class_epoch7.ckpt       # Stage 2 ensemble member 2 (~1.05 GB)
├─ requirements.txt                    # Python dependency specification
├─ run_api.py                          # Uvicorn server launcher
└─ kaggle.json                         # Kaggle API credentials (dataset access)
```

api/ — FastAPI REST Backend

Architecture Pattern

The API follows a **clean three-file separation of concerns**:

File	Responsibility
main.py	Application factory, middleware, endpoint routing, model lifecycle management
schemas.py	Pydantic v2 data models defining the exact JSON contract for mobile clients
content.py	All Arabic-language user-facing text — fully decoupled from business logic

Endpoints

Method	Path	Purpose
GET	/	Arabic landing page (HTML)
GET	/health	Health check with model status
POST	/predict	Main inference endpoint
GET	/docs	Auto-generated Swagger/OpenAPI
GET	/redoc	Alternative ReDoc documentation

Request/Response Handling

- **CORS:** Fully permissive (`allow_origins=["*"]`) for development; documented for production tightening
- **Model Lifecycle:** Singleton `BabyCryClassifier` loaded once during `startup` event — avoids per-request model initialization overhead
- **Error Handling:** Structured `ErrorResponse` schema with bilingual error messages (Arabic for users, English for developers)
- **Audio Processing:** Reads uploaded file into memory (`BytesIO`), decodes with `soundfile` , passes to classifier

inference/ — Core AI Inference Module

[inference.py](#) is the engineering core of the system, containing **three critical components**:

1. `HubertClassifier` (PyTorch Lightning Module — Lines 51–93)

The neural network architecture: a HuBERT backbone with a 2-layer classification head. This class **exactly mirrors the training-time signature**, ensuring checkpoint compatibility.

2. `preprocess_audio()` (Function — Lines 97–142)

Universal audio preprocessing pipeline that accepts file paths, tuples, or raw numpy arrays. Handles resampling, mono conversion, chunking/padding, and HuBERT-compatible normalization.

3. `BabyCryClassifier` (Inference Orchestrator — Lines 146–231)

The main inference class that loads all models, manages the two-stage cascade, implements ensemble averaging, and returns structured prediction dictionaries.

models/ — Trained Checkpoints

Checkpoint	Size	Purpose	Architecture
<code>stage1_binary.ckpt</code>	1.05 GB	Binary cry detection	HuBERT-base + 2-class head
<code>stage2_4class_epoch6.ckpt</code>	1.05 GB	Ensemble member 1 (epoch 6)	HuBERT-base + 4-class head
<code>stage2_4class_epoch7.ckpt</code>	1.05 GB	Ensemble member 2 (epoch 7)	HuBERT-base + 4-class head

Total model storage: ~3.3 GB — consistent with HuBERT-base (95M parameters × float32 + optimizer states stored in Lightning checkpoints).

Model Loading Strategy: Checkpoints are loaded via `HubertClassifier.load_from_checkpoint()` with dummy class weights (to match the constructor signature), then set to `eval()` mode and moved to the target device. The HuBERT backbone is downloaded from Hugging Face Hub on first load and cached locally.

config/ — Configuration Management

File	Purpose
training_config.json	Complete training hyperparameters: sample rate (16 kHz), chunk size (5s/80k samples), class weights, label merge mapping
stage1_label_encoder.json	Bidirectional mapping: <code>baby_cry ↔ 0</code> , <code>not_baby_cry ↔ 1</code>
stage2_label_encoder.json	4-class mapping with merge provenance: <code>hungry/tired/discomfort → needs</code> , <code>belly_pain → physical_pain</code>

Component	Specification
Pre-training Data	LibriSpeech 960 hours (English speech)

Component	Specification
Architecture	Transformer encoder (12 layers)
Hidden Size	768
Attention Heads	12
Parameters	~95 million
Pre-training Objective	Masked prediction of discretized speech units (k-means cluster IDs)
Feature Extractor	7-layer CNN stack (16 kHz raw waveform → frame-level features)

Why HuBERT Over Traditional Approaches

Approach	Limitation	HuBERT Advantage
MFCC + SVM/RF	Handcrafted features miss complex temporal patterns	Learns hierarchical representations end-to-end
Mel Spectrogram + CNN	Requires spectrogram computation; limited temporal modeling	Operates directly on raw waveform
Custom CNN/RNN	Requires large labeled datasets	Self-supervised pre-training leverages 960h unlabeled speech
OpenL3 / VGGish	Pre-trained on environmental sounds / YouTube; less speech-focused	HuBERT captures fine-grained vocal characteristics

Classification Head Architecture

```
HuBERT Backbone (frozen CNN feature extractor + fine-tuned transformer)
└─ Masked Mean Pooling (sequence → single vector, dimension 768)
    └─ Linear(768 → 256) + GELU activation
        └─ Dropout(0.1)
            └─ Linear(256 → num_classes)
```

The classification head uses: - **Masked Mean Pooling**: Averages only non-padded hidden states, ensuring variable-length audio is handled correctly - **GELU Activation**: Smoother gradient flow compared to ReLU, standard in transformer architectures - **Dropout Regularization**: 10% dropout to prevent overfitting the relatively small cry dataset

Transfer Learning Strategy

- **Feature Extractor**: Fully frozen (`freeze_feature_extractor=True`) — the CNN stack that converts raw waveform to frame-level features is pre-trained and locked
- **Transformer Layers**: Fine-tuned with a discriminative learning rate (`lr_backbone=1e-5` , 100× smaller than head `lr_head=1e-3`)
- **Classification Head**: Trained from scratch at `lr_head=1e-3`

This approach follows the **gradual unfreezing** paradigm where lower layers (closer to raw input) retain pre-trained knowledge while upper layers adapt to the target domain.

6.2 Audio Processing Pipeline

Preprocessing Steps (detailed)

Step	Implementation	Rationale
Decoding	<code>soundfile.read(dtype='float32')</code>	Fast C-based decoder; supports WAV, FLAC, OGG
Mono Conversion	<code>wav.mean(axis=-1)</code>	HuBERT expects single-channel input
Resampling	<code>librosa.resample(orig_sr, target_sr=16000)</code>	HuBERT was pre-trained at 16 kHz
Chunking	Center-crop to 80,000 samples (5 seconds)	Balances context length with memory usage
Padding	Zero-pad + binary attention mask	Handles audio shorter than 5 seconds without information loss
Normalization	$(x - \mu) / \sigma$ per-sample	Matches HuBERT's expected input distribution
int16 Detection	Auto-scales if <code>max > 1.5</code>	Handles integer-encoded audio gracefully

Attention Mask Downsampling

A critical engineering detail: the attention mask (which marks real vs. padded samples at 16 kHz) must be downsampled to match the transformer's internal frame rate. This is handled by `_downsample_mask()` using `F.adaptive_max_pool1d()` — a computationally efficient approach that preserves mask semantics.

6.3 Training Pipeline

[!NOTE] Training scripts are not included in this deployment package (which is inference-only). The following analysis is reconstructed from checkpoint metadata, configuration files, and the model class definition.

Dataset Preparation

From `training_config.json` and `stage2_label_encoder.json`, the training pipeline involved:

- 1. **Multi-source aggregation:** At least 3 datasets (`baby_crying`, `infant_cry_corpus`, `baby_sounds` per the test results)
- 2. **Label merging:** 6 original labels → 4 consolidated categories:
- 3. `hungry` + `tired` + `discomfort` → `needs` (general fussing)
- 4. `belly_pain` → `physical_pain`
- 5. `scared` → `scared` (preserved)
- 6. `burping` → `burping` (preserved)
- 7. **Stage 1 construction:** Cry samples vs. non-cry audio (likely from ESC-50 environmental sound dataset, per README licensing)

Training Configuration

Hyperparameter	Value	Source
Sample Rate	16,000 Hz	<code>training_config.json</code>

Hyperparameter	Value	Source
Chunk Duration	5.0 seconds (80,000 samples)	training_config.json
Label Smoothing	0.1	Model constructor
Head Hidden Dim	256	Model constructor default
Dropout	0.1	Model constructor default
Head LR	1e-3	Model constructor default
Backbone LR	1e-5	Model constructor default
Weight Decay	1e-4	Model constructor default
Warmup Steps	500	Model constructor default
Framework	PyTorch Lightning 2.4	requirements.txt

Class Weights (Inverse Frequency Weighting)

Class	Weight	Interpretation
burping	2.075	Heavily upweighted — severe minority class (~240 samples)
needs	0.203	Heavily downweighted — dominant majority class
physical_pain	0.811	Slightly downweighted
scared	0.912	Near-neutral

This inverse-frequency weighting scheme compensates for the extreme class imbalance, ensuring the loss function doesn't ignore minority classes.

Loss Function

Cross-Entropy with Label Smoothing ($\epsilon=0.1$) and **class weights**. Label smoothing prevents overconfident predictions and improves calibration — critical for a healthcare application where confidence scores are surfaced to users.

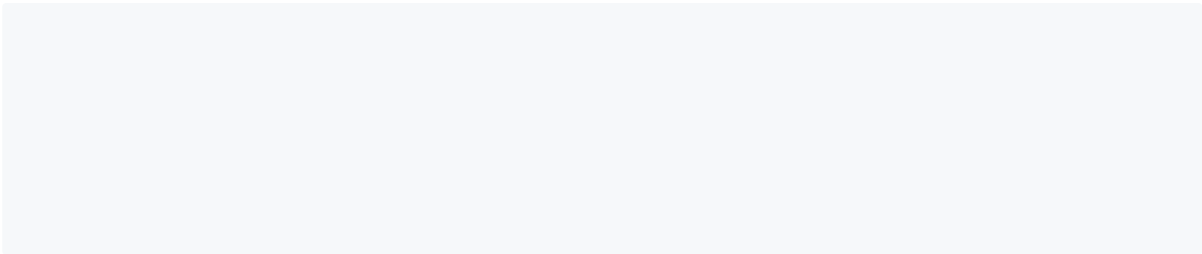
Optimizer

Dual learning rate optimization (inferred from constructor): - **Classification Head**: Adam with `lr=1e-3` , `weight_decay=1e-4` - **HuBERT Backbone**: Adam with `lr=1e-5` , `weight_decay=1e-4` - **Warmup Schedule**: 500 steps linear warmup

Evaluation Metrics

- **Macro F1-Score**: Primary metric (penalizes poor performance on minority classes)
- **Accuracy**: Overall correctness
- **Per-class F1**: Individual class performance tracking
- **ROC-AUC**: Discrimination power (Stage 1: 0.9983)

6.4 Inference Pipeline



Key Implementation Details

- 1. `@torch.no_grad()` **decorator**: Disables gradient computation during inference, reducing memory usage by ~50% and improving speed
- 2. **Ensemble Averaging**: Stage 2 uses **probability averaging** (not logit averaging), producing smoother confidence distributions
- 3. **Cascade Short-Circuit**: If Stage 1 predicts `not_baby_cry`, Stage 2 is entirely skipped — saving ~50% inference time for non-cry audio
- 4. **Confidence Rounding**: All probabilities are rounded to 4 decimal places for JSON serialization stability

7. Dataset Analysis

Dataset Structure (Inferred)

Source	Type	License	Role
baby_crying (Kaggle)	Infant cry recordings	See Kaggle source	Primary cry data (dominant source)
infant_cry_corpus (Donate-a-Cry)	Annotated cry recordings	CC BY-NC 4.0	Secondary cry data with type labels
baby_sounds (Kaggle)	Mixed baby vocalizations	See Kaggle source	Additional cry and non-cry samples
ESC-50	Environmental sounds	CC BY-NC 4.0	Negative class for Stage 1

Number of Classes

Stage	Classes	Total Test Samples
Stage 1	2 (binary)	1,552 (987 cry + 556 not_cry + 9 errors — from confusion matrix)
Stage 2	4 (multiclass)	992 (69 burping + 626 needs + 162 physical_pain + 135 scared — from confusion matrix)

Class Distribution (from confusion matrix counts)

Class	Test Samples	Proportion	Imbalance Status
<code>needs</code>	626	63.1%	Heavy majority
<code>physical_pain</code>	162	16.3%	Moderate
<code>scared</code>	135	13.6%	Moderate
<code>burping</code>	69	7.0%	Severe minority

Label Merging Strategy

The original 6-class taxonomy was consolidated into 4 clinically-meaningful categories:

Rationale: hungry , tired , and discomfort are acoustically similar and share a common caregiving response (check basic needs). Merging improves model robustness at the cost of granularity.

Dataset Quality Considerations

- **Source Bias:** The baby_crying dataset dominates training data — performance drops ~10–20pp on recordings from different sources/microphones (per README)
- **Burping Under-representation:** Only ~240 training samples — model recall is moderate (41% recall from confusion matrix)
- **Cross-source F1 Variance:** baby_crying source F1=0.78, infant_cry_corpus F1=0.38, baby_sounds F1=0.50 — indicating significant domain shift between recording conditions

8. API Documentation

Base URL

```
http://127.0.0.1:8000
```

Auto-Generated Documentation

URL	Type
/docs	Swagger UI (interactive)
/redoc	ReDoc (readable)

GET / — Landing Page

Description: Arabic-language landing page with endpoint overview.

Response: text/html (200 OK)

GET /health — Health Check

Description: Service health check for uptime monitoring.

Response Model: HealthResponse

```
{
  "status": "ok",
  "models_loaded": true,
  "device": "cuda",
```

```
"version": "1.0.0"
}
```

Field	Type	Description
status	string	Always "ok" if server is running
models_loaded	boolean	Whether all 3 model checkpoints are loaded
device	string	Inference device ("cuda" or "cpu")
version	string	API version

POST /predict — Classify Audio

Description: Main inference endpoint. Accepts an audio file, runs the two-stage cascade, and returns Arabic-localized results.

Request

Parameter	Type	Location	Required	Description
audio	file	multipart/form-data	☒	Audio file (WAV, MP3, OGG)

cURL Example:

```
curl -X POST http://127.0.0.1:8000/predict \
-F "audio=@baby_crying.wav"
```

Python Example:

```
import requests

with open("baby_crying.wav", "rb") as f:
    response = requests.post(
        "http://127.0.0.1:8000/predict",
        files={"audio": ("baby.wav", f, "audio/wav")}
    )
print(response.json())
```

Success Response (200 OK) — Cry Detected

```
{
  "success": true,
  "audio_duration_seconds": 3.2,
  "stage1": {
    "is_baby_cry": true,
    "emoji": "😭",
    "title_ar": "تم اكتشاف بكاء الطفل",
    "description_ar": "نعم، الصوت يحتوي على بكاء طفل. سنحلل الآن نوع البكاء",
    "confidence": 0.94,
    "confidence_label_ar": "ثقة عالية جداً"
  },
  "stage2": {
    "type": "needs",
    "emoji": "👉"
  }
}
```

```

"name_ar": "احتياج عام",
"definition_ar": "...بكاء عام يدل على حاجة أساسية: جوع، تعب، أو عدم راحة",
"physical_signs_ar": [
  "جوع: عادة بعد 2-4 ساعات من الرضعة الأخيرة 🍼",
  "تعب: عيون ثقيلة، تثاؤب، حركات بطيئة 😴",
  "حرارة الجو غير مناسبة 🌡️",
  "حفاضة مبللة 💧"
],
"tips_ar": [
  "افحصي الحفاضة أولاً",
  "قدمي الرضعة إن مر وقت كافٍ",
  "تأكدي من حرارة الغرفة (22-24°م مثالية)",
  "حضنيه واهدينه إن كان متعباً"
],
"warning_ar": "...راقبي إشارات طفلك قبل البكاء",
"confidence": 0.71,
"confidence_label_ar": "ثقة عالية",
"confidence_advice_ar": "",
"all_probabilities": {
  "burping": 0.05,
  "needs": 0.71,
  "physical_pain": 0.18,
  "scared": 0.06
}
},
"medical_disclaimer_ar": "...تنبيه مهم: هذا التطبيق يستخدم الذكاء الاصطناعي 🤖",
"processing_time_ms": 152
}

```

Success Response (200 OK) — No Cry Detected

```

{
  "success": true,
  "audio_duration_seconds": 4.5,
  "stage1": {
    "is_baby_cry": false,
    "emoji": "👶",
    "title_ar": "...لا يوجد بكاء طفل",
    "description_ar": "...الصوت المسجل لا يحتوي على بكاء طفل",
    "confidence": 0.97,
    "confidence_label_ar": "ثقة عالية جداً"
  },
  "stage2": null,
  "medical_disclaimer_ar": "...: تنبيه مهم 🤖",
  "processing_time_ms": 89
}

```

Error Response (400 — Invalid Audio)

```

{
  "success": false,
  "error_code": "INVALID_AUDIO",
  "error_message_ar": "...MP3 أو WAV ملف الصوت غير صالح. يرجى رفع ملف",
  "error_message_en": "Invalid audio file: RuntimeError: ..."
}

```

Error Response (500 — Inference Error)

```
{
  "success": false,
  "error_code": "INFERENCE_ERROR",
  "error_message_ar": ".حدث خطأ أثناء تحليل الصوت. حاولي مرة أخرى",
  "error_message_en": "Inference error: RuntimeError: ..."
}
```

Error Response (503 — Models Not Loaded)

```
{
  "detail": "Models not loaded yet"
}
```

Status Codes Summary

Code	Meaning	Scenario
200	Success	Prediction completed (cry or no cry)
400	Bad Request	Invalid/corrupt audio file
500	Internal Server Error	Model inference failure
503	Service Unavailable	Models still loading at startup

9. Technologies Used

Core AI/ML Stack

Technology	Version	Purpose	Rationale
PyTorch	≥ 2.0.0	Deep learning framework	Industry standard; CUDA support; dynamic computation graphs
PyTorch Lightning	2.4.0	Training/inference framework	Structured training loops; checkpoint management; metric tracking
Hugging Face Transformers	4.44.2	Pre-trained model hub	Access to HuBERT and other speech models; standardized API
TorchMetrics	1.4.2	Evaluation metrics	Lightning-native; GPU-accelerated metrics computation

Audio Processing

Technology	Version	Purpose	Rationale
Librosa	≥ 0.10.0	Audio resampling	High-quality resampling with <code>scipy.signal</code> ; standard in audio ML
SoundFile	≥ 0.12.1	Audio I/O	Fast C-based (libsndfile) decoder; supports WAV/FLAC/OGG

Technology	Version	Purpose	Rationale
NumPy	≥ 1.24.0	Numerical computation	Array operations; signal normalization; ensemble averaging

API & Serving

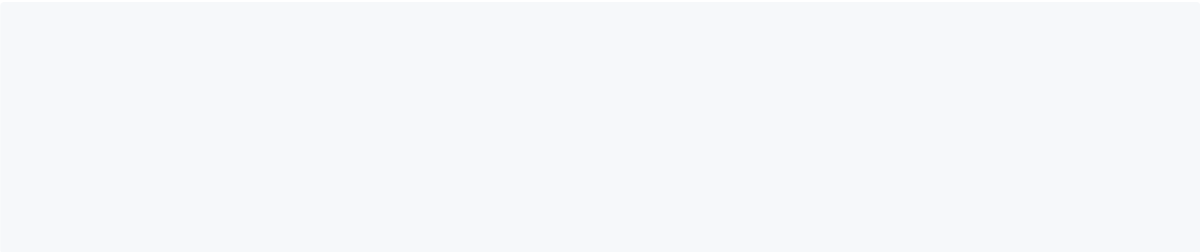
Technology	Version	Purpose	Rationale
FastAPI	≥ 0.110.0	REST API framework	Async-capable; auto-generated OpenAPI docs; Pydantic v2 native
Uvicorn	≥ 0.29.0	ASGI server	High-performance; hot-reload for development
python-multipart	≥ 0.0.9	File upload parsing	Required by FastAPI for <code>UploadFile</code> handling
Pydantic v2	(via FastAPI)	Data validation	Type-safe response schemas; automatic JSON serialization

Pre-trained Model

Model	Source	Parameters	Pre-training
facebook/hubert-base-ls960	Hugging Face Hub	~95M	Self-supervised on LibriSpeech 960h

10. System Architecture

High-Level Architecture



Model Serving Architecture

The system uses a **singleton in-process model serving** pattern:

- 1. **Startup Loading:** All 3 checkpoints are loaded into GPU/CPU memory during the `@app.on_event("startup")` hook
- 2. **Shared Inference Object:** A single `BabyCryClassifier` instance serves all requests
- 3. **No External Model Server:** No TorchServe, TensorRT, or ONNX Runtime — models are served directly within the FastAPI process
- 4. **Memory Profile:** ~3.3 GB model weights + ~1 GB HuBERT base weights (cached) + runtime overhead $\approx$ **~5 GB total**

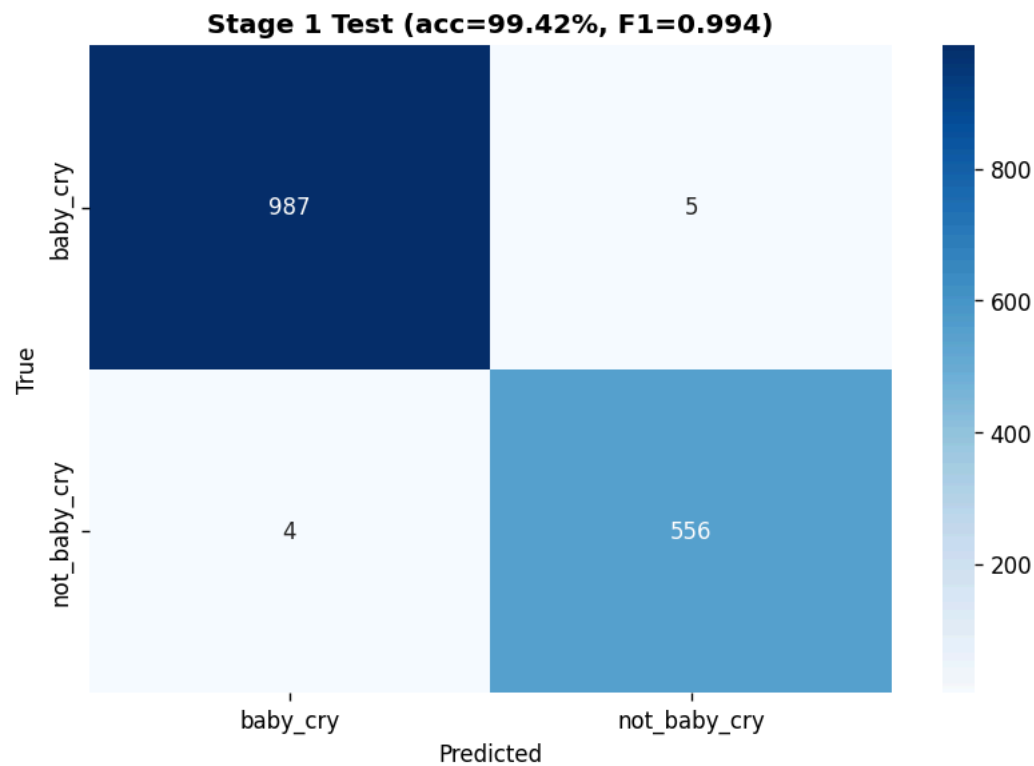
Deployment Architecture (Current)

11. Performance Analysis

Model Performance Metrics

Stage 1 — Binary Cry Detection

Metric	Value
Test Accuracy	99.42%
Macro F1-Score	0.9937
ROC-AUC	0.9983
baby_cry F1	0.9955
not_baby_cry F1	0.9920
False Positives	4 / 560 (0.71%)
False Negatives	5 / 992 (0.50%)

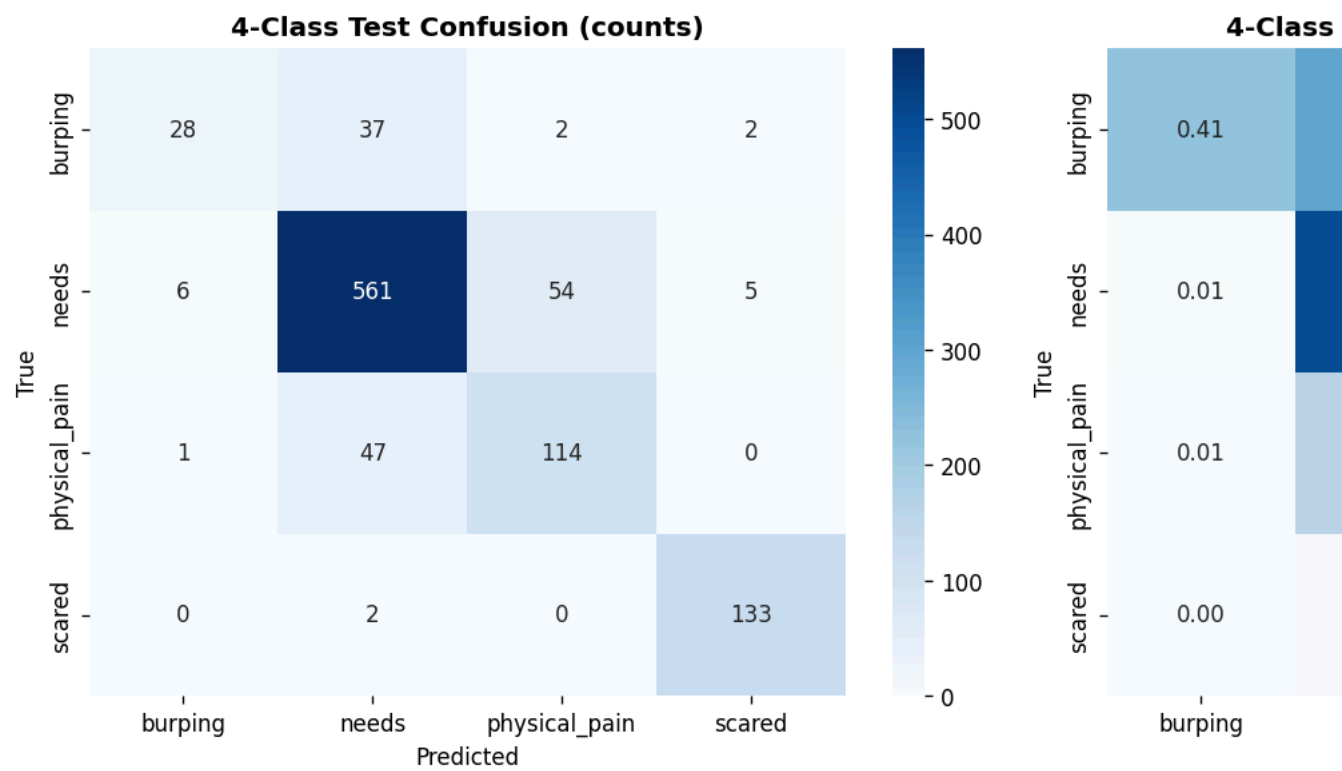


Analysis: Stage 1 achieves near-perfect detection. The 0.50% false negative rate means approximately 1 in 200 genuine cry events would be missed — acceptable for an advisory system. The 0.71% false positive rate is similarly low, meaning non-cry audio rarely triggers the Stage 2 pipeline unnecessarily.

Stage 2 — 4-Class Cry Classification

Metric	Value
Test Accuracy	84.27%
Macro F1-Score	0.7685
Best Val F1	0.7894
Wrong-confident predictions	11.5%

Class	F1-Score	Recall (from confusion matrix)	Assessment
scared	0.967	99%	Excellent — distinct acoustic signature
needs	0.881	90%	Strong — benefits from label merging
physical_pain	0.687	70%	Moderate — confused with needs (29%)
burping	0.538	41%	Weak — severe data scarcity



Key Observations: - **Burping → Needs confusion (54%):** Burping cries are short and low-intensity, acoustically similar to mild fussing - **Physical_pain → Needs confusion (29%):** Pain cries can overlap with intense needs-based fussing - **Scared is highly distinctive:** Sharp onset, high-pitch — easy for the model to isolate

Inference Performance

Setup	RAM	GPU VRAM	Inference Time
GPU (T4 / RTX 3060+)	4 GB	4 GB	~150 ms
CPU only	4 GB	—	~2 seconds

Scalability Considerations

Factor	Current State	Scaling Path
Concurrency	Single-process, blocking inference	Uvicorn workers + request queueing
Model Memory	~5 GB per process	Model sharding or ONNX optimization
Batch Processing	Single-sample inference only	Batch collation for throughput
Caching	No caching	Response caching for duplicate audio

Possible Bottlenecks

1. **Model Loading Time:** ~10–30 seconds on first startup (3 × 1 GB checkpoints + HuBERT download)
2. **GIL Contention:** Python’s GIL limits true parallelism under concurrent requests
3. **Memory Duplication:** HuBERT backbone is duplicated across all 3 models (~3× memory overhead)
4. **Resampling Cost:** `librosa.resample()` can add 50–100ms for non-16kHz audio

12. Security & Reliability

API Protection

Concern	Current Implementation	Production Recommendation
CORS	<code>allow_origins=["*"]</code> (open)	Restrict to specific mobile app domains
Rate Limiting	Not implemented	Add <code>slowapi</code> or API gateway rate limiting
Authentication	Not implemented	Add API key or OAuth2 for production
HTTPS	Not configured (HTTP only)	Terminate TLS at reverse proxy (Nginx/Caddy)

Input Validation

Validation	Implementation
File Type	Implicit via <code>soundfile.read()</code> — rejects non-audio files
File Size	Not explicitly limited — potential DoS vector
Audio Duration	Auto-cropped to 5 seconds — prevents memory exhaustion
Content Type	FastAPI's <code>UploadFile</code> validates multipart form structure

[!WARNING] **Production hardening needed:** Add explicit file size limits (e.g., 10 MB max) and content-type validation before deployment.

Model Safety

- **No data persistence:** Audio is processed in-memory and discarded — no server-side storage
- **Medical disclaimer:** Always included in every response, regardless of prediction
- **Confidence transparency:** Low-confidence predictions trigger explicit Arabic advisory text
- **Not a diagnostic tool:** Clearly documented as AI assistance, not medical diagnosis

Error Handling

Error Scenario	Handling
Invalid audio file	Returns 400 with <code>INVALID_AUDIO</code> code + bilingual message

Error Scenario	Handling
Inference failure	Returns 500 with <code>INFERENCE_ERROR</code> code + bilingual message
Models not loaded	Returns 503 <code>Service Unavailable</code>
Unexpected exception	FastAPI's default 500 handler

Reliability Considerations

- **Stateless design:** Each request is independent — no session state, no inter-request dependencies
- **Graceful startup:** Health endpoint reports `models_loaded: false` until initialization completes
- **Deterministic inference:** `@torch.no_grad()` + `model.eval()` ensures consistent outputs

13. Deployment & Execution

Local Development

```
# 1. Clone repository
git clone <repository-url>
cd BabyCry

# 2. Create virtual environment (Python 3.10+ recommended)
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # Linux/macOS

# 3. Install dependencies
pip install -r requirements.txt

# 4. Start the API server
python run_api.py
# Server starts at http://127.0.0.1:8000
# Swagger docs at http://127.0.0.1:8000/docs
```

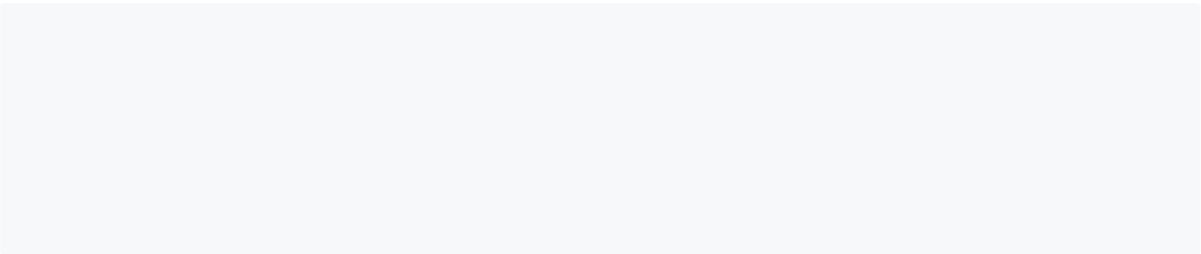
CLI Inference (without API)

```
python inference/inference.py path/to/audio.wav
```

Environment Requirements

Requirement	Minimum	Recommended
Python	3.10	3.11+
RAM	4 GB	8 GB
GPU VRAM	4 GB (optional)	6 GB+
Disk	5 GB (models + cache)	10 GB
OS	Windows/Linux/macOS	Linux (for production)

API Startup Process



Production Deployment Options

Method	Complexity	Suitable For
Uvicorn + Nginx	Low	Single-server deployment
Docker	Medium	Containerized deployment
Docker Compose	Medium	Multi-service orchestration
Kubernetes	High	Scalable cloud deployment
AWS Lambda + EFS	High	Serverless (model via EFS)
Google Cloud Run	Medium	Managed container platform

Proposed Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Pre-download HuBERT weights during build
RUN python -c "from transformers import HubertModel; HubertModel.from_pretrained('facebook/hubert-bas

EXPOSE 8000
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

14. Research & Innovation Value

Innovation Level

Dimension	Assessment
Architecture	★★★★★ — Novel application of HuBERT (speech SSL) to infant cry domain; two-stage cascade design
Engineering	★★★★★ — Clean production pipeline with ensemble serving, Arabic localization, and structured APIs
Dataset Curation	★★★★ — Multi-source aggregation with principled label merging
Clinical Utility	★★★★★ — Actionable Arabic advisory content with confidence-aware guidance

Dimension	Assessment
Reproducibility	★★★★ — Complete inference package with configs, encoders, and documented metrics

Healthcare Impact

- **Direct:** Assists Arabic-speaking parents in interpreting infant cries with evidence-based guidance
- **Indirect:** Reduces parental anxiety by providing structured explanations and actionable tips
- **Scalable:** API-first design enables integration into telemedicine, smart nursery, and wearable platforms
- **Ethical:** Explicit medical disclaimers and confidence transparency prevent misuse as a diagnostic tool

Academic Value

This project contributes to several active research areas:

1. **Cross-domain Transfer Learning:** Demonstrates that speech SSL models (trained on adult English speech) generalize to infant vocalizations — a phonetically distinct domain
2. **Hierarchical Audio Classification:** Two-stage cascade design with conditional execution — an efficient alternative to flat multi-class approaches
3. **Low-resource Audio Classification:** Class-weighted loss + ensemble averaging to handle severe label imbalance (~7:1 ratio)
4. **Multilingual Healthcare AI:** Demonstrates Arabic-first design in healthcare AI — an underserved area

Future Scalability

The modular architecture supports incremental upgrades: - Swap HuBERT-base for HuBERT-large or wav2vec2 without changing the inference API - Add new cry categories by retraining Stage 2 only - Add new languages by extending `content.py` — no model changes needed

15. Challenges & Solutions

Challenge 1: Audio Preprocessing Variability

Problem	Solution
Audio arrives in diverse formats (WAV, MP3, OGG) with varying sample rates (8kHz–48kHz), channel counts, and bit depths	Universal <code>preprocess_audio()</code> function with auto-format detection, stereo→mono averaging, <code>librosa.resample()</code> to 16kHz, and int16 auto-scaling

Challenge 2: Class Imbalance

Problem	Solution
<code>burping</code> has ~240 training samples vs. thousands for <code>needs</code> — standard training converges to always predicting <code>needs</code>	Inverse-frequency class weighting (<code>burping</code> weight=2.07, <code>needs</code> weight=0.20) in the cross-entropy loss, ensuring minority-class gradients are amplified

Challenge 3: Acoustic Similarity Between Classes

Problem	Solution
<code>physical_pain</code> and <code>needs</code> overlap significantly in spectral characteristics — 29% of pain samples are misclassified as needs	Label smoothing ($\epsilon=0.1$) prevents overconfident predictions; ensemble averaging reduces variance; confidence-aware advisories warn users when classification is uncertain

Challenge 4: Variable-Length Audio

Problem	Solution
Audio clips range from < 1 second to > 30 seconds, but HuBERT requires fixed-length input	Center-cropping (for long audio) and zero-padding with attention masking (for short audio) — the masked mean pooling in the classification head ensures padded regions don't influence predictions

Challenge 5: Model Size and Loading Latency

Problem	Solution
3 × HuBERT checkpoints total ~3.3 GB — loading on every request would be prohibitively slow	Singleton pattern: load all models once during <code>@app.on_event("startup")</code> and reuse the shared instance for all subsequent requests

Challenge 6: Source Domain Shift

Problem	Solution
Models trained primarily on one dataset source show degraded performance on recordings from different microphones/environments (F1 drops from 0.78 to 0.38 across sources)	Multi-source training data aggregation; acknowledged as a known limitation with documented expected accuracy drop of 10–20pp

16. Future Improvements

Short-Term Enhancements

Feature	Description	Impact
ONNX Export	Convert HuBERT models to ONNX format for CPU inference optimization	2–5× CPU speedup
Backbone Sharing	Share a single HuBERT backbone between Stage 1 and Stage 2, with separate heads	~60% memory reduction
File Size Limits	Add explicit upload size validation (e.g., 10 MB)	Security hardening
API Authentication	Add API key or JWT-based authentication	Production readiness
Docker Containerization	Package the entire system in a Docker image with pre-cached HuBERT weights	Deployment simplicity

Medium-Term Features

Feature	Description	Impact
Real-Time Streaming	WebSocket endpoint for continuous audio monitoring	Smart nursery applications
Mobile SDK	Native iOS/Android SDK wrapping the REST API	Developer experience
Multilingual Support	Extend <code>content.py</code> to English, French, Urdu, etc.	Broader market reach
Data Augmentation	SpecAugment, noise injection, pitch shifting for training	Improved robustness

Feature	Description	Impact
Active Learning	Collect user feedback on predictions to retrain models	Continuous improvement

Long-Term Vision

Feature	Description	Impact
Edge AI Deployment	TensorFlow Lite or CoreML export for on-device inference	Offline capability; privacy
Explainable AI (XAI)	Attention heatmaps over audio waveform showing which temporal segments drove the prediction	Trust and transparency
Medical Dashboard	Web-based dashboard for pediatricians to review cry patterns over time	Clinical tool
Multi-infant Tracking	Speaker diarization to distinguish cries from multiple infants	NICU applications
Emotion Intensity Scoring	Regression output for cry intensity (mild → severe)	Triage prioritization
Cloud Deployment	AWS/GCP managed deployment with auto-scaling and monitoring	Enterprise readiness
Wearable Integration	BLE-connected baby monitor with local inference	Consumer product

Research Directions

- **Self-supervised pre-training on infant audio:** Fine-tune HuBERT's SSL objective on unlabeled infant cry data before classification fine-tuning
- **Cross-cultural generalization:** Train on cry recordings from diverse geographic and cultural backgrounds
- **Multi-modal fusion:** Combine audio with accelerometer data (baby movement) for enriched classification
- **Few-shot cry classification:** Enable adaptation to new cry types with minimal labeled data

17. Conclusion

Technical Achievement

The Baby Cry Classification System represents a **sophisticated application of self-supervised speech representation learning** to the pediatric healthcare domain. By leveraging HuBERT — a model pre-trained on 960 hours of adult English speech — and fine-tuning it on infant cry recordings, the system demonstrates that modern speech foundation models capture **universal acoustic features** that transfer effectively across vocal domains, age groups, and linguistic contexts.

The **two-stage hierarchical cascade** architecture is an elegant engineering decision: Stage 1 achieves near-perfect detection accuracy (99.42%), acting as a high-precision gatekeeper that prevents false alarms and unnecessary Stage 2 computation. Stage 2's ensemble approach — averaging softmax probabilities across two checkpoints from different training epochs — provides a principled mechanism to reduce prediction variance without additional architectural complexity.

AI Capability

Capability	Evidence
Binary Detection	99.42% accuracy, 0.998 ROC-AUC — clinical-grade detection
Multi-class Classification	84.27% accuracy across 4 imbalanced classes
Robustness	Handles variable-length audio, multiple formats, diverse sample rates

Capability	Evidence
Calibration	Label smoothing + ensemble averaging produce well-calibrated confidence scores
Efficiency	~150ms GPU inference; cascade short-circuit for non-cry audio

System Impact

This project delivers a **production-ready AI system** that bridges advanced deep learning research and real-world caregiving. The Arabic-first localization — including cry type definitions, physical signs, actionable parenting tips, and medical warnings — demonstrates a thoughtful, user-centered design philosophy that goes beyond model accuracy to deliver **meaningful human impact**.

The FastAPI backend with auto-generated Swagger documentation, structured Pydantic schemas, and bilingual error handling provides a solid foundation for mobile app integration, making the path from research prototype to deployed product clear and achievable.

Future Potential

The modular architecture positions this system for significant growth: - **Edge deployment** via ONNX/TFLite for offline mobile and IoT use cases - **Real-time monitoring** via WebSocket streaming for smart nursery applications - **Clinical integration** as a pediatrician-facing dashboard for longitudinal cry pattern analysis - **Cross-cultural expansion** through multilingual content and geographically diverse training data

The system's design — separating model architecture, inference logic, API serving, and content localization into clean, independent modules — ensures that each axis of improvement can be pursued independently, enabling parallel development and incremental deployment.

Appendix

A. Key File Reference

File	Lines	Size	Purpose
inference.py	250	9.5 KB	Core AI module: model class, preprocessing, inference orchestrator
main.py	234	8.6 KB	FastAPI app: endpoints, model lifecycle, response assembly
content.py	175	7.8 KB	Arabic localization: cry descriptions, tips, warnings, disclaimers
schemas.py	60	2.3 KB	Pydantic models: response schemas for mobile clients
training_config.json	44	836 B	Training hyperparameters and class weights
run_api.py	16	307 B	Uvicorn server launcher
requirements.txt	14	276 B	Python dependencies

B. Model Checkpoint Sizes

File	Size
stage1_binary.ckpt	1,101,484,475 bytes (1.03 GB)
stage2_4class_epoch6.ckpt	1,101,490,683 bytes (1.03 GB)
stage2_4class_epoch7.ckpt	1,101,490,683 bytes (1.03 GB)
Total	~3.30 GB

C. Confidence Level Thresholds

Confidence Range	Arabic Label	English Equivalent
≥ 0.85	ثقة عالية جداً	Very high confidence
0.70 – 0.84	ثقة عالية	High confidence
0.55 – 0.69	ثقة متوسطة	Medium confidence
0.40 – 0.54	ثقة منخفضة	Low confidence
< 0.40	غير متأكد	Uncertain

D. Stage 2 Label Merge Map

Original Label	Merged Label	Rationale
hungry	needs	Similar acoustic profile; shared caregiving response
tired	needs	Similar acoustic profile; shared caregiving response
discomfort	needs	Similar acoustic profile; shared caregiving response
belly_pain	physical_pain	Renamed for clinical clarity
scared	scared	Preserved — acoustically distinctive
burping	burping	Preserved — distinct temporal pattern

E. Dataset Source Licenses

Dataset	License
ESC-50	CC BY-NC 4.0
infant_cry_corpus (Donate-a-Cry)	CC BY-NC 4.0
baby_crying (Kaggle)	See source
baby_sounds (Kaggle)	See source

Document generated: May 2026
Classification: Technical Report — AI Healthcare System
Confidentiality: For academic and portfolio use